

Politechnika Warszawska
Wydział Elektroniki i Technik Informacyjnych

PSIR PROJEKT

GRA PENNEY'S ANTE

Autorzy:
ZESPÓŁ:
ADAM GOŚCINIAK
NATALIA ŚLEPOWROŃSKA
KAMILA KWAŚNIEWSKA

Warszawa 2024Z

Spis treści

1. Cel projektu	2
2. Założenia	2
2.1. Architektura systemu	2
2.1.1. Schematy blokowe	3
2.2. Struktury danych	5
2.2.1. Pliki środowiskowe	6
3. Nasz protokół ALP	7
3.1. Połączenie węzłów z serwerem - wiadomość Hello	7
3.2. Informowanie serwera o wygranej	8
3.3. Przesłanie wzorca	9
3.4. Niezawodność protokołu	9
4. Serwer - komunikaty na konsoli	10
5. Wyniki	11
6. Podsumowanie	12

1. Cel projektu

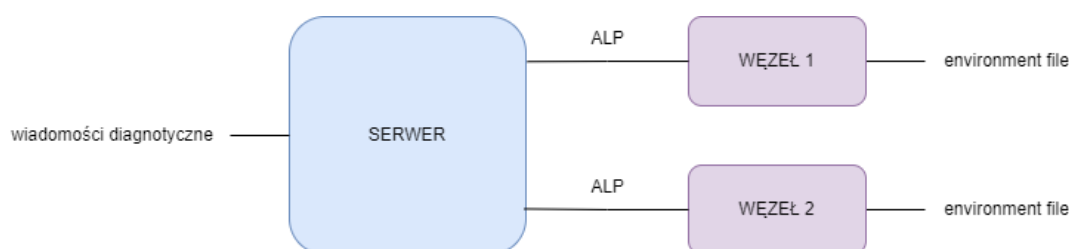
Celem projektu było opracowanie własnego protokołu ALP bazującego na protokole UDP, dzięki któremu serwer oraz węzły IoT (symulatory Arduino) mogą zagrać ze sobą w grę Penney's ante.

2. Założenia

Gracze oraz serwer operują na maszynach wirtualnych udostępnionych nam w ramach przedmiotu PSIR. Komunikują się one ze sobą dzięki opracowanemu przez nas protokołowi ALP. Gra Penney's ante jest grą binarną bazującą na tworzeniu sekwencji wyników rzutów monetą przez dwóch graczy. Gracze wybierają sekwencje wyników o tej samej długości - w naszym przypadku wzorzec jest trzy bitowy. Dalsza część gry to rzut monetą do momentu aż sekwencja któregoś z graczy się nie pojawi. Gracz którego sekwencja pojawiła się jako pierwsza wygrywa.

2.1. Architektura systemu

Program serwera został napisany w języku C, zaś algorytmy węzłów zostały przygotowane w środowisku Arduino IDE zgodnie z wymaganiami stawianymi przed nami przez Arduino i jego emulator. Korzystaliśmy jedynie z podstawowych bibliotek oraz bibliotek przygotowanych w ramach przedmiotu PSIR.



Rysunek 1: Architektura systemu

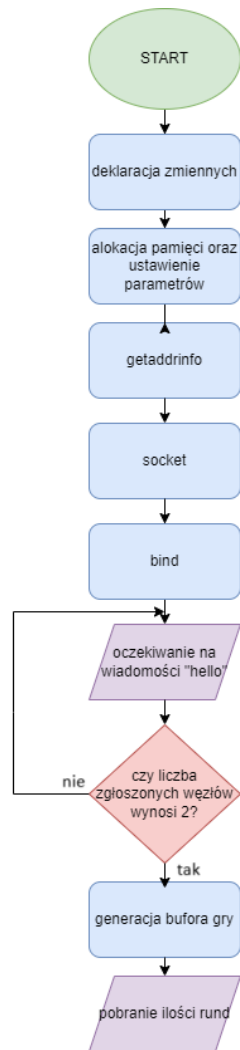
Wszystkie maszyny posiadają ustawioną kartę sieciową typu bridged. W naszym przypadku, elementy systemu posiadają następujące IP:

- serwer - 192.168.56.119
- node 1 - 192.168.56.120
- node 2 - 192.168.56.121

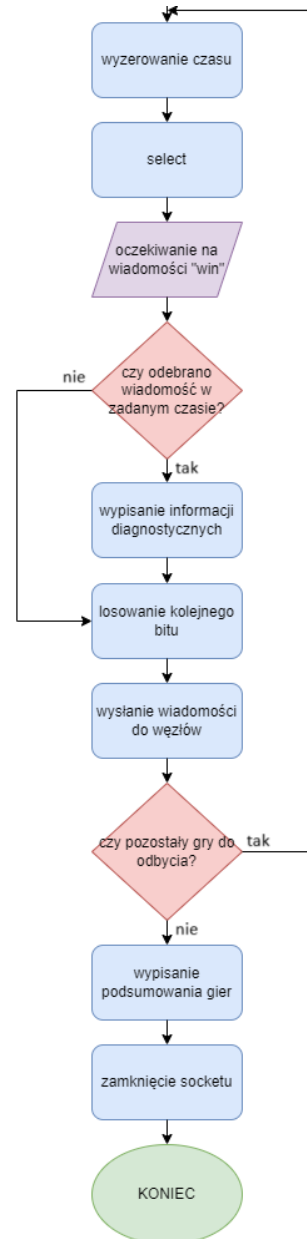
Protokół IP jest niezbędny do działania naszego protokołu ALP. Port po którym zachodzi komunikacja to 1234.

2.1.1. Schematy blokowe

Poniżej przedstawiamy schemat blokowy algorytmu działania serwera:



Rysunek 2: Część I

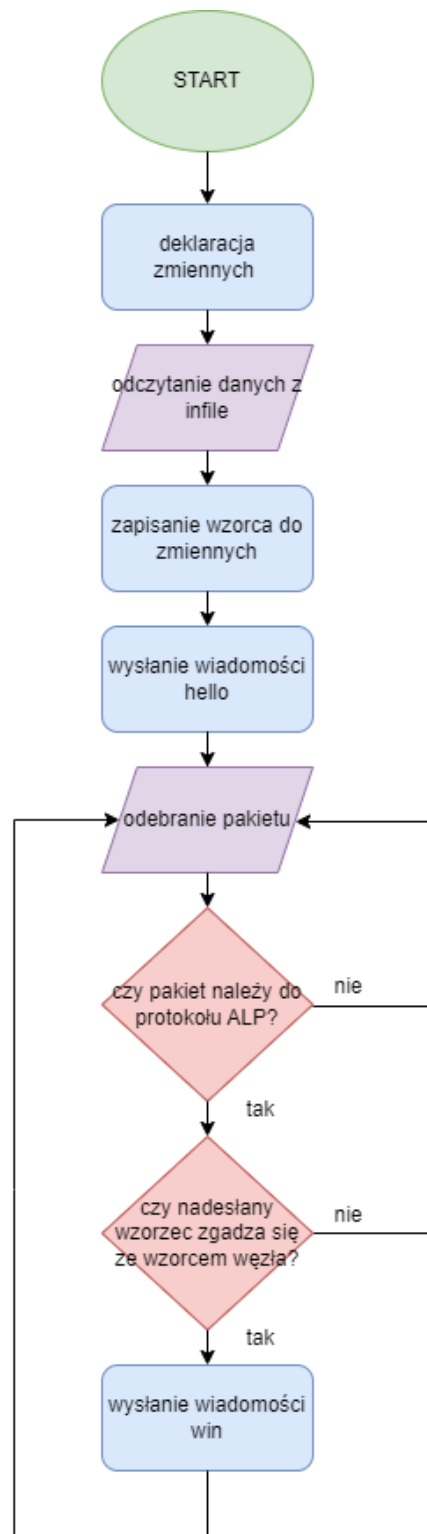


Rysunek 3: Część II

Skorzystaliśmy ze wszystkich niezbędnych funkcji dla serwera UDP:

- **socket** - stworzenie socketu przez który będzie odbywać się komunikacja
- **bind** - ustawia lokalny adres socketu
- **recvfrom** - nasłuchiwanie wiadomości i ich odbieranie
- **sendto** - wysyłanie wiadomości
- **close** - zamknięcie socketu

Schemat blokowy algorytmu węzłów przedstawia się następująco:



Rysunek 4: schemat dla węzłów

2.2. Struktury danych

Jedynie kod serwera zawiera napisane przez nas funkcje, które posiadają następujące funkcjonalności:

- **game_buff_gen** - generuje on bufor gry wraz z nagłówkiem, dwoma pierwszymi rzutami i bitami kontrolnymi
- **game** - generuje nowe losowanie i uzupełnia wzorzec
- **print_pattern** - wypisuje na konsoli obecny wzorzec
- **print_game_sum** - wypisuje informacje diagnostyczne po wygranej
- **print_game_sum_all** - wypisuje informacje diagnostyczne po zakończeniu działania programu

Wartym odnotowania są także zadeklarowane przez nas w kodzie serwera zmienne służące do zliczania danych:

- **serv_number** - liczba zgłoszonych serwerów
- **game_number** - liczba odbytych gier
- **max_game** - maksymalna liczba gier
- **win_node1** - liczba wygranych dla node 1
- **win_node2** - liczba wygranych dla node 2
- **bit_node1** - liczba losowań potrzebnych do wygranej w danej grze dla node 1
- **bit_node2** - liczba losowań potrzebnych do wygranej w danej grze dla node 2
- **count** - liczba odbytych losowań

Dodatkowo korzystamy z buforów typu `char` do odbioru i przesyłu wiadomości oraz ze struktur **sockaddr_in** i **addrinfo** do przechowywania danych potrzebnych do działania funkcji sieciowych.

Kolejnymi zmiennymi są zmienne o typie `struct timeval` oraz **fd_set** potrzebne są do wywołania funkcji **select** - obserwatora deskryptorów. Dzięki tej funkcji rozwiązujemy problem nieskończonego nasłuchiwanie na wiadomości.

2.2.1. Pliki środowiskowe

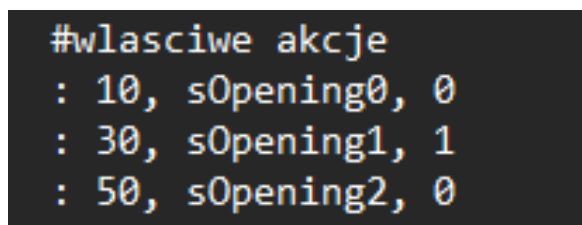
Pliki środowiskowe służą nam do emulacji wprowadzenia danych do symulatora Arduino. Dzięki niemu możemy ustawić konkretne wartości dla pinów cyfrowych i analogowych.

```
1 #Opis zasobow
2 + sOpening0,status,D0      # input  0=OPEN, 1=CLOSED
3 + sOpening1,status,D1      # input  0=OPEN, 1=CLOSED
4 + sOpening2,status,D2      # input  0=OPEN, 1=CLOSED
5 + sOpening3,status,D3      # input  0=OPEN, 1=CLOSED
6 + sOpening4,status,D4      # input  0=OPEN, 1=CLOSED
7 + sOpening5,status,D5      # input  0=OPEN, 1=CLOSED
8 + sOpening6,status,D6      # input  0=OPEN, 1=CLOSED
9 + sOpening7,status,D7      # input  0=OPEN, 1=CLOSED
10 + sOpening8,status,D8     # input  0=OPEN, 1=CLOSED
11 + sOpening9,status,D9     # input  0=OPEN, 1=CLOSED
12 + sOpening10,status,D10   # input  0=OPEN, 1=CLOSED
13 + sOpening11,status,D11   # input  0=OPEN, 1=CLOSED
14 + sOpening12,status,D12   # input  0=OPEN, 1=CLOSED
15 + sOpening13,status,D13   # input  0=OPEN, 1=CLOSED
16
17 #opis akcji
18
19 #inicjowanie stanu poczatkowego
20 : 0, sOpening0, 0
21 : 0, sOpening1, 0
22 : 0, sOpening2, 0
23 : 0, sOpening3, 0
24 : 0, sOpening4, 0
25 : 0, sOpening5, 0
26 : 0, sOpening6, 0
27 : 0, sOpening7, 0
28 : 0, sOpening8, 0
29 : 0, sOpening9, 0
30 : 0, sOpening10, 0
31 : 0, sOpening11, 0
32 : 0, sOpening12, 0
33 : 0, sOpening13, 0
34
35 #wlasciwe akcje
36 : 10, sOpening0, 0
37 : 30, sOpening1, 1
38 : 50, sOpening2, 0
39 #: ##uwaga jest bledem jezeli ostatnia linia nie jest pusta lub komentarzem
```

Listing 1: infile.txt

Właściwe akcje oznaczają aktywację diod logicznych D0, D1, D2, których wartości determinowane są w pliku wejściowym "infile.txt".

Przykładowa sekwencja dla węzła pierwszego: **010**.



```
#wlasciwe akcje
: 10, sOpening0, 0
: 30, sOpening1, 1
: 50, sOpening2, 0
```

Rysunek 5: infile



```
GPIO -----
Z0:0x03ff  Z1:0x03f
D0:0  D1:1  D2:0  D
UART -----
```

Rysunek 6: node 1

3. Nasz protokół ALP

Struktura naszego protokołu przedstawia się następująco:

Typ wiadomości	Nagłówek	Typ informacji	Bity kontrolne	Długość wiadomości
Hello	00	identyfikator węzła (0 / 1)	101	6 bitów
Ze wzorcem	01	3 bitowy wzorzec	101	8 bitów
Win	10	identyfikator węzła (0 / 1)	101	6 bitów

Tabela 1: ALP

3.1. Połączenie węzłów z serwerem - wiadomość Hello

Węzły w celu nawiązania połączenia z serwerem wysyłają wiadomość "Hello" na znany adres IP serwera, składającą się z sekwencji bitów **000101** dla węzła 1 oraz **001101** dla węzła 2.

```
1 char hello[] = "000101";
2 ...
3 Udp.begin(localPort);
4 packetSize = Udp.parsePacket();
5 Udp.beginPacket(servIp, servPort);
6 Udp.write(hello);
7 Udp.endPacket();
8 Udp.begin(localPort);
9 ...
```

Listing 2: Kod węzła 1

```
1 char hello[] = "001101";
2 ...
3 Udp.begin(localPort);
4 packetSize = Udp.parsePacket();
5 Udp.beginPacket(servIp, servPort);
6 Udp.write(hello);
7 Udp.endPacket();
8 Udp.begin(localPort);
9 ...
```

Listing 3: Kod węzła 2

```
1 printf("START NASLUCHIWANIA -----\\n");
2 while(serv_number < MAX_SERV){
3     //rec -----\\n
4     mess = recvfrom(sock, buffer, MAX_BUFF, 0, (struct sockaddr*)&c, &c_len);
5     if (mess == -1) {
6         printf("ERROR: %s (%s:%d)\\n", strerror(errno), __FILE__, __LINE__);
7         exit(-1);
8     }
9     buffer[mess]='\\0';
10    if(buffer[0] == '0' && buffer[1] == '0' && buffer[3] == '1' && buffer[4] == '0' \\
11        && buffer[5] == '1'){
12        if(buffer[2] == '0'){
13            memcpy(&servers[0], &c, c_len);
14            printf("odebrano hello od node 1: %s\\n",buffer);
15        }
16        else if(buffer[2] == '1'){
17            memcpy(&servers[1], &c, c_len);
18            printf("odebrano hello od node 2: %s\\n",buffer);
19        }
20        serv_number = serv_number + 1;
21    }
```

Listing 4: Kod serwera

3.2. Informowanie serwera o wygranej

Serwer jest tylko przestrzenią do gry - niczym moneta podczas takiej gry w prawdziwym życiu, to nie on decyduje o wygranej, tylko jest informowany przez któryś z węzłów który rozpozna swój wzorec pośród losowanych liczb w celu zakończenia danej rundy gry. Wiadomość informująca o wygranej w węźle 1 to **100101**, a w węźle 2 **101101**.

```
1 ...
2 char win[] = "100101";
3 ...
4 if(packetBuffer[2] == z2 && packetBuffer[3] == z1 && packetBuffer[4] == z0){
5     delay(100);
6     packetSize = Udp.parsePacket();
7     Udp.beginPacket(servIp, servPort);
8     Udp.write(win);
9     Udp.endPacket();
10 ...
```

Listing 5: Kod węzła 1

```
1 char win[] = "101101";
2 ...
3 if(packetBuffer[2] == z2 && packetBuffer[3] == z1 && packetBuffer[4] == z0){
4     delay(100);
5     packetSize = Udp.parsePacket();
6     Udp.beginPacket(servIp, servPort);
7     Udp.write(win);
8     Udp.endPacket();
9 ...
```

Listing 6: Kod węzła 2

```
1 ...
2     else if(sel>0){
3         //rec -----
4         mess = recvfrom(sock, buffer, MAX_BUFFER, 0, (struct sockaddr*)&c, &c_len);
5         if (mess == -1) {
6             printf("ERROR: %s (%s:%d)\n", strerror(errno), __FILE__, __LINE__);
7             exit(-1);
8         }
9         buffer[mess]='\0';
10        if(buffer[0] == '1' && buffer[1] == '0' && buffer[3] == '1' \
11            && buffer[4] == '0' && buffer[5] == '1'){
12            if(buffer[2] == '0'){
13                game_number = game_number + 1;
14                win_node1 = win_node1 + 1;
15                print_game_sum(buffer, count, game_number, win_node1, bit_node1);
16                bit_node1 = 0;
17                bit_node2 = 0;
18            }
19            else if(buffer[2] == '1'){
20                game_number = game_number + 1;
21                win_node2 = win_node2 + 1;
22                print_game_sum(buffer, count, game_number, win_node2, bit_node2);
23                bit_node1 = 0;
24                bit_node2 = 0;
25            }
26        }
27 ...
```

Listing 7: Kod Serwera

3.3. Przesłanie wzorca

Serwer cyklicznie, maksymalnie co 1 sekundę przesyła do obu węzłów aktualny wzorec, który jest generowany przez funkcję `game`.

```
1 ...
2     else{
3         //send -----
4         game(game_buff);
5         print_pattern(game_buff);
6         bit_node1 = bit_node1 + 1;
7         bit_node2 = bit_node2 + 1;
8         count = count + 1;
9         snprintf(buffer, MAX_BUFFER, "%s", game_buff);
10        mess = sendto(sock, buffer, MAX_BUFFER, 0, (const struct sockaddr*)&servers[0], c_len);
11        if (mess == -1) {
12            printf("ERROR: %s (%s:%d)\n", strerror(errno), __FILE__, __LINE__);
13            exit(-1);
14        }
15        mess = sendto(sock, buffer, MAX_BUFFER, 0, (const struct sockaddr*)&servers[1], c_len);
16        if (mess == -1) {
17            printf("ERROR: %s (%s:%d)\n", strerror(errno), __FILE__, __LINE__);
18            exit(-1);
19        }
20    }
21 ...
```

Listing 8: kod serwera

3.4. Niezawodność protokołu

Ponieważ protokół UDP nie gwarantuje nam pełnej niezawodności dodaliśmy bity kontrolne które są sprawdzane w węzłach i na serwerze. Niezawodność naszego protokołu ALP nie jest całkowita, nie udało nam się wyeliminować zjawiska gubienia pakietów z serwera do węzłów.

4. Serwer - komunikaty na konsoli

Poniżej zaprezentowane są komunikaty serwera podczas różnych etapów gry:

```
student@iot:~$ ./server
START NASLUCHIWANIA -----
_
```

Rysunek 7: Włączenie serwera

Serwer komunikuje rozpoczęcie działania oraz nasłuchiwanie na podłączenie się węzłów.

```
student@iot:~$ ./server
START NASLUCHIWANIA -----
odebrano hello od node 1: 000101
odebrano hello od node 2: 001101
PODAJ LICZBE GIER -----
```

Rysunek 8: Nawiązanie połączenia z węzłami

Serwer komunikuje że węzły nawiązały komunikację oraz prosi użytkownika o wpisanie liczby gier jakie mają zostać przeprowadzone.

```
START GRY -----
wzorzec: 1 0 1
wzorzec: 0 1 1
wzorzec: 1 1 1
wzorzec: 1 1 1
wzorzec: 1 1 0
wzorzec: 1 0 0
wzorzec: 0 0 1
wzorzec: 0 1 1
wzorzec: 1 1 0
INFO DIAGNOSTYCZNE -----
odebrano win od node 2: 101101
liczba gier 1
liczba wygranych: 1
liczba losowań do wygranej: 11
prawdopodobieństwo wystąpienia danego wzorca: 0.111111
estymowana liczba rzutów do wystąpienia danego wzorca: 11.000000
-----
```

Rysunek 9: Gra

W trakcie gry wyświetlany jest generowany wzorzec, następnie komunikat który węzeł wygrał grę oraz informacje, takie jak która to gra z kolei, ile gier dany węzeł wygrał, ile losowań zostało przeprowadzonych przed wygraną oraz estymaty prawdopodobieństwa wystąpienia danego wzorca i liczby rzutów do wygranej, które są co grę aktualizowane.

```
PODSUMOWANIE node 1 -----
liczba gier 5
liczba wygranych: 2
liczba losowań: 32
prawdopodobieństwo wystąpienia wzorca: 0.066667
estymowana liczba rzutów do wystąpienia wzorca: 16.000000
PODSUMOWANIE node 2 -----
liczba gier 5
liczba wygranych: 3
liczba losowań: 32
prawdopodobieństwo wystąpienia wzorca: 0.100000
estymowana liczba rzutów do wystąpienia wzorca: 10.666667
student@iot:~$ _
```

Rysunek 10: Podsumowanie i estymaty po 5 grach

Po rozegraniu wszystkich gier następuje podsumowanie całości dla obu węzłów. Ile gier zostało rozegrane, liczby wygranych oraz estymaty na podstawie wszystkich rozegranych gier.

5. Wyniki

Uruchomiliśmy nasz program dla 1000 gier. Wzorzec dla node 1 ustawiliśmy na **010**, a dla node 2 na **110**.

```

PODSUMOWANIE node 1 -----
liczba gier 1000
liczba wygranych: 494
liczba losowan: 5241
prawdopodobienstwo wystapienia wzorca: 0.094293
estymowana liczba rzutow do wystapienia wzorca: 10.609312
-----

PODSUMOWANIE node 2 -----
liczba gier 1000
liczba wygranych: 506
liczba losowan: 5241
prawdopodobienstwo wystapienia wzorca: 0.096583
estymowana liczba rzutow do wystapienia wzorca: 10.357708
-----

```

Rysunek 11: Wyniki

Prawdopodobieństwo wystąpienia wzorca liczymy na podstawie wzoru $P = \frac{\text{liczbawygranych}}{\text{liczbarzutow} - \text{dugoscwzorca} + 1}$. Liczbę rzutów potrzebnych do wystąpienia wzorca liczymy na podstawie wzoru $E = \frac{\text{liczbarzutow}}{\text{liczbawygranych}}$. Wiemy, że kolejne rzuty monetą są niezależne i w 3 rzutach monetą wystąpienie któregośkolwiek z możliwych wzorców jest tak samo prawdopodobne ($P = \frac{1}{8}$).

Jednak w przypadku gry Penney's ante prawdopodobieństwo wygranej danego wzorca nie jest tak oczywiste. Można je policzyć z wykorzystaniem liczb wiodących Conway'a. Wykonaliśmy obliczenia dla zadanych wzorców:

$$\begin{array}{l}
 \omega_1 = 010 \quad \omega_2 = 110 \\
 C^{\omega_1, \omega_1} \qquad C^{\omega_2, \omega_2} \\
 \begin{array}{ccc}
 010 & 010 & 010 \\
 010 & 01 & 0 \\
 \hline
 1 & 10 & 101
 \end{array}
 \qquad
 \begin{array}{ccc}
 110 & 110 & 110 \\
 110 & 10 & 1 \\
 \hline
 1 & 11 & 110
 \end{array} \\
 C^{\omega_2, \omega_1} \qquad C^{\omega_1, \omega_2} \\
 \begin{array}{ccc}
 110 & 110 & 110 \\
 010 & 01 & 0 \\
 \hline
 0 & 00 & 001
 \end{array}
 \qquad
 \begin{array}{ccc}
 010 & 010 & 010 \\
 110 & 11 & 1 \\
 \hline
 0 & 00 & 000
 \end{array} \\
 C^{\omega_1, \omega_1} = 5 \quad C^{\omega_2, \omega_2} = 6 \quad C^{\omega_2, \omega_1} = 1 \quad C^{\omega_1, \omega_2} = 0 \\
 \frac{P_1}{P_2} = \frac{C^{\omega_2, \omega_2} - C^{\omega_2, \omega_1}}{C^{\omega_1, \omega_1} - C^{\omega_1, \omega_2}} = \frac{6 - 1}{5 - 0} = 1 \Rightarrow P_1 = P_2
 \end{array}$$

Rysunek 12: Prawdopodobieństwo wygranej

Prawdopodobieństwo wygranej wzorca **010** jest równe temu dla wzorca **110**, co zgadza się z otrzymanymi przez nas wynikami (są one bardzo podobne dla obu węzłów).

Nie będzie tak jednak dla każdej pary wzorców. Rzuty monetą są niezależne, jednak aktualny wzorzec zależy od poprzednich rzutów.

Przykładowo:

$$\begin{array}{l}
 \omega_3 = 111 \quad \omega_4 = 001 \\
 \begin{array}{c} C_{\omega_3, \omega_3} \\
 \begin{array}{ccc}
 111 & 111 & 111 \\
 111 & 11 & 1 \\
 \hline
 1 & 11 & 111
 \end{array}
 \end{array}
 \quad
 \begin{array}{c} C_{\omega_4, \omega_4} \\
 \begin{array}{ccc}
 001 & 001 & 001 \\
 001 & 00 & 0 \\
 \hline
 1 & 10 & 100
 \end{array}
 \end{array} \\
 \begin{array}{c} C_{\omega_3, \omega_4} \\
 \begin{array}{ccc}
 001 & 001 & 001 \\
 111 & 11 & 1 \\
 \hline
 0 & 00 & 001
 \end{array}
 \end{array}
 \quad
 \begin{array}{c} C_{\omega_4, \omega_3} \\
 \begin{array}{ccc}
 111 & 111 & 111 \\
 001 & 000 & 0 \\
 \hline
 0 & 00 & 000
 \end{array}
 \end{array} \\
 \frac{P_3}{P_4} = \frac{4-1}{4-0} = \frac{3}{4} \Rightarrow P_3 = \frac{3}{7} \cdot P_4
 \end{array}$$

Rysunek 13: Prawdopodobieństwo wygranej - wzorce 111 i 001

Ponieważ $P_3 + P_4 = 1$ to $P_3 = \frac{3}{10}$ a $P_4 = \frac{7}{10}$. Uruchomiliśmy nasz program dla 200 gier. Wzorec dla węzła 1 ustawiliśmy na 111, a dla węzła 2 na 001.

```

estymowana liczba rzutów do wystąpienia danego wzorca: 11.031579
-----
PODSUMOWANIE node 1 -----
liczba gier 200
liczba wygranych: 95
liczba losowań: 1048
prawdopodobieństwo wystąpienia wzorca: 0.090822
estymowana liczba rzutów do wystąpienia wzorca: 11.031579
-----
PODSUMOWANIE node 2 -----
liczba gier 200
liczba wygranych: 105
liczba losowań: 1048
prawdopodobieństwo wystąpienia wzorca: 0.100382
estymowana liczba rzutów do wystąpienia wzorca: 9.980952
-----
student@iot:~$

```

Rysunek 14: Wyniki

Widzimy, że wzorec dla którego prawdopodobieństwo wygranej było większe, wygrywał częściej.

6. Podsumowanie

Podczas pracy nad projektem mieliśmy możliwość podsumować, zaimplementować ale i również pogłębić wiedzę zdobytą podczas trwania przedmiotu, szczególnie odnośnie projektowania systemów opartych na protokole UDP oraz węzłami IoT. Projekt ten również odświeżył nam w ciekawy sposób wiedzę odnośnie prawdopodobieństwa i umożliwił poznanie nowych, ciekawych zagadnień zarówno z dziedziny programowania, jak i matematyki.